

Programming Challenges

1. Date

Design a class called `Date`. The class should store a date in three integers: `month`, `day`, and `year`. There should be member functions to print the date in the following forms:

12/25/2014

December 25, 2014

25 December 2014

Demonstrate the class by writing a complete program implementing it.

Input Validation: Do not accept values for the day greater than 31 or less than 1. Do not accept values for the month greater than 12 or less than 1.

2. Employee Class

Write a class named `Employee` that has the following member variables:

- **name**. A string that holds the employee's name.
- **idNumber**. An `int` variable that holds the employee's ID number.
- **department**. A string that holds the name of the department where the employee works.
- **position**. A string that holds the employee's job title.

The class should have the following constructors:

- A constructor that accepts the following values as arguments and assigns them to the appropriate member variables: employee's name, employee's ID number, department, and position.
- A constructor that accepts the following values as arguments and assigns them to the appropriate member variables: employee's name and ID number. The `department` and `position` fields should be assigned an empty string (`""`).
- A default constructor that assigns empty strings (`""`) to the `name`, `department`, and `position` member variables, and 0 to the `idNumber` member variable.

Write appropriate mutator functions that store values in these member variables and accessor functions that return the values in these member variables. Once you have written the class, write a separate program that creates three `Employee` objects to hold the following data.

Name	ID Number	Department	Position
Susan Meyers	47899	Accounting	Vice President
Mark Jones	39119	IT	Programmer
Joy Rogers	81774	Manufacturing	Engineer

The program should store this data in the three objects and then display the data for each employee on the screen.

3. Car Class

Write a class named `Car` that has the following member variables:

- **yearModel**. An `int` that holds the car's year model.
- **make**. A string that holds the make of the car.
- **speed**. An `int` that holds the car's current speed.



VideoNote
Solving the
Employee
Class Problem

In addition, the class should have the following constructor and other member functions.

- **Constructor.** The constructor should accept the car's year model and make as arguments. These values should be assigned to the object's `yearModel` and `make` member variables. The constructor should also assign 0 to the `speed` member variables.
- **Accessor.** Appropriate accessor functions to get the values stored in an object's `yearModel`, `make`, and `speed` member variables.
- **accelerate.** The `accelerate` function should add 5 to the `speed` member variable each time it is called.
- **brake.** The `brake` function should subtract 5 from the `speed` member variable each time it is called.

Demonstrate the class in a program that creates a `Car` object, and then calls the `accelerate` function five times. After each call to the `accelerate` function, get the current speed of the car and display it. Then, call the `brake` function five times. After each call to the `brake` function, get the current speed of the car and display it.

4. Personal Information Class

Design a class that holds the following personal data: name, address, age, and phone number. Write appropriate accessor and mutator functions. Demonstrate the class by writing a program that creates three instances of it. One instance should hold your information, and the other two should hold your friends' or family members' information.

5. RetailItem Class

Write a class named `RetailItem` that holds data about an item in a retail store. The class should have the following member variables:

- **description.** A string that holds a brief description of the item.
- **unitsOnHand.** An `int` that holds the number of units currently in inventory.
- **price.** A `double` that holds the item's retail price.

Write a constructor that accepts arguments for each member variable, appropriate mutator functions that store values in these member variables, and accessor functions that return the values in these member variables. Once you have written the class, write a separate program that creates three `RetailItem` objects and stores the following data in them.

	Description	Units On Hand	Price
Item #1	Jacket	12	59.95
Item #2	Designer Jeans	40	34.95
Item #3	Shirt	20	24.95

6. Inventory Class

Design an `Inventory` class that can hold information and calculate data for items in a retail store's inventory. The class should have the following *private* member variables:

Variable Name	Description
<code>itemNumber</code>	An <code>int</code> that holds the item's item number.
<code>quantity</code>	An <code>int</code> for holding the quantity of the items on hand.
<code>cost</code>	A <code>double</code> for holding the wholesale per-unit cost of the item
<code>totalCost</code>	A <code>double</code> for holding the total inventory cost of the item (calculated as <code>quantity</code> times <code>cost</code>).

The class should have the following *public* member functions:

Member Function	Description
Default Constructor	Sets all the member variables to 0.
Constructor #2	Accepts an item's number, cost, and quantity as arguments. The function should copy these values to the appropriate member variables and then call the <code>setTotalCost</code> function.
<code>setItemNumber</code>	Accepts an integer argument that is copied to the <code>itemNumber</code> member variable.
<code>setQuantity</code>	Accepts an integer argument that is copied to the <code>quantity</code> member variable.
<code>setCost</code>	Accepts a <code>double</code> argument that is copied to the <code>cost</code> member variable.
<code>setTotalCost</code>	Calculates the total inventory cost for the item (<code>quantity</code> times <code>cost</code>) and stores the result in <code>totalCost</code> .
<code>getItemNumber</code>	Returns the value in <code>itemNumber</code> .
<code>getQuantity</code>	Returns the value in <code>quantity</code> .
<code>getCost</code>	Returns the value in <code>cost</code> .
<code>getTotalCost</code>	Returns the value in <code>totalCost</code> .

Demonstrate the class in a driver program.

Input Validation: Do not accept negative values for item number, quantity, or cost.

7. TestScores Class

Design a `TestScores` class that has member variables to hold three test scores. The class should have a constructor, accessor, and mutator functions for the test score fields and a member function that returns the average of the test scores. Demonstrate the

class by writing a separate program that creates an instance of the class. The program should ask the user to enter three test scores, which are stored in the `TestScores` object. Then the program should display the average of the scores, as reported by the `TestScores` object.

8. Circle Class

Write a `Circle` class that has the following member variables:

- `radius`: a double
- `pi`: a double initialized with the value 3.14159

The class should have the following member functions:

- **Default Constructor.** A default constructor that sets `radius` to 0.0.
- **Constructor.** Accepts the radius of the circle as an argument.
- **setRadius.** A mutator function for the radius variable.
- **getRadius.** An accessor function for the radius variable.
- **getArea.** Returns the area of the circle, which is calculated as
$$\text{area} = \text{pi} * \text{radius} * \text{radius}$$
- **getDiameter.** Returns the diameter of the circle, which is calculated as
$$\text{diameter} = \text{radius} * 2$$
- **getCircumference.** Returns the circumference of the circle, which is calculated as
$$\text{circumference} = 2 * \text{pi} * \text{radius}$$

Write a program that demonstrates the `Circle` class by asking the user for the circle's radius, creating a `Circle` object, and then reporting the circle's area, diameter, and circumference.

9. Population

In a population, the birth rate and death rate are calculated as follows:

$$\text{Birth Rate} = \text{Number of Births} \div \text{Population}$$

$$\text{Death Rate} = \text{Number of Deaths} \div \text{Population}$$

For example, in a population of 100,000 that has 8,000 births and 6,000 deaths per year, the birth rate and death rate are:

$$\text{Birth Rate} = 8,000 \div 100,000 = 0.08$$

$$\text{Death Rate} = 6,000 \div 100,000 = 0.06$$

Design a `Population` class that stores a population, number of births, and number of deaths for a period of time. Member functions should return the birth rate and death rate. Implement the class in a program.

Input Validation: Do not accept population figures less than 1, or birth or death numbers less than 0.

10. Number Array Class

Design a class that has an array of floating-point numbers. The constructor should accept an integer argument and dynamically allocate the array to hold that many numbers. The destructor should free the memory held by the array. In addition, there should be member functions to perform the following operations:

- Store a number in any element of the array
- Retrieve a number from any element of the array

- Return the highest value stored in the array
- Return the lowest value stored in the array
- Return the average of all the numbers stored in the array

Demonstrate the class in a program.

11. Payroll

Design a `PayRoll` class that has data members for an employee's hourly pay rate, number of hours worked, and total pay for the week. Write a program with an array of seven `PayRoll` objects. The program should ask the user for the number of hours each employee has worked and will then display the amount of gross pay each has earned.

Input Validation: Do not accept values greater than 60 for the number of hours worked.

12. Coin Toss Simulator

Write a class named `Coin`. The `Coin` class should have the following member variable:

- A string named `sideUp`. The `sideUp` member variable will hold either “heads” or “tails” indicating the side of the coin that is facing up.

The `Coin` class should have the following member functions:

- A default constructor that randomly determines the side of the coin that is facing up (“heads” or “tails”) and initializes the `sideUp` member variable accordingly.
- A void member function named `toss` that simulates the tossing of the coin. When the `toss` member function is called, it randomly determines the side of the coin that is facing up (“heads” or “tails”) and sets the `sideUp` member variable accordingly.
- A member function named `getSideUp` that returns the value of the `sideUp` member variable.

Write a program that demonstrates the `Coin` class. The program should create an instance of the class and display the side that is initially facing up. Then, use a loop to toss the coin 20 times. Each time the coin is tossed, display the side that is facing up. The program should keep count of the number of times heads is facing up and the number of times tails is facing up, and display those values after the loop finishes.

13. Tossing Coins for a Dollar

For this assignment, you will create a game program using the `Coin` class from Programming Challenge 12. The program should have three instances of the `Coin` class: one representing a quarter, one representing a dime, and one representing a nickel.

When the game begins, your starting balance is \$0. During each round of the game, the program will toss the simulated coins. When a coin is tossed, the value of the coin is added to your balance if it lands heads-up. For example, if the quarter lands heads-up, 25 cents is added to your balance. Nothing is added to your balance for coins that land tails-up. The game is over when your balance reaches \$1 or more. If your balance is exactly \$1, you win the game. You lose if your balance exceeds \$1.

14. Fishing Game Simulation

For this assignment, you will write a program that simulates a fishing game. In this game, a six-sided die is rolled to determine what the user has caught. Each possible item is worth a certain number of fishing points. The points will not be displayed until

the user has finished fishing, and then a message is displayed congratulating the user depending on the number of fishing points gained.

Here are some suggestions for the game's design:

- Each round of the game is performed as an iteration of a loop that repeats as long as the player wants to fish for more items.
- At the beginning of each round, the program will ask the user whether he or she wants to continue fishing.
- The program simulates the rolling of a six-sided die (use the `Die` class that was demonstrated in this chapter).
- Each item that can be caught is represented by a number generated from the die. For example, 1 for “a huge fish,” 2 for “an old shoe,” 3 for “a little fish,” and so on.
- Each item the user catches is worth a different amount of points.
- The loop keeps a running total of the user's fishing points.
- After the loop has finished, the total number of fishing points is displayed, along with a message that varies depending on the number of points earned.

15. Mortgage Payment

Design a class that will determine the monthly payment on a home mortgage. The monthly payment with interest compounded monthly can be calculated as follows:

$$\text{Payment} = \frac{\text{Loan} \times \frac{\text{Rate}}{12} \times \text{Term}}{\text{Term} - 1}$$

where

$$\text{Term} = \left(1 + \frac{\text{Rate}}{12}\right)^{12 \times \text{Years}}$$

Payment = the monthly payment

Loan = the dollar amount of the loan

Rate = the annual interest rate

Years = the number of years of the loan

The class should have member functions for setting the loan amount, interest rate, and number of years of the loan. It should also have member functions for returning the monthly payment amount and the total amount paid to the bank at the end of the loan period. Implement the class in a complete program.

Input Validation: Do not accept negative numbers for any of the loan values.

16. Freezing and Boiling Points

The following table lists the freezing and boiling points of several substances.

Substance	Freezing Point	Boiling Point
Ethyl Alcohol	-173	172
Oxygen	-362	-306
Water	32	212

Design a class that stores a temperature in a `temperature` member variable and has the appropriate accessor and mutator functions. In addition to appropriate constructors, the class should have the following member functions:

- **isEthylFreezing.** This function should return the `bool` value `true` if the temperature stored in the `temperature` field is at or below the freezing point of ethyl alcohol. Otherwise, the function should return `false`.
- **isEthylBoiling.** This function should return the `bool` value `true` if the temperature stored in the `temperature` field is at or above the boiling point of ethyl alcohol. Otherwise, the function should return `false`.
- **isOxygenFreezing.** This function should return the `bool` value `true` if the temperature stored in the `temperature` field is at or below the freezing point of oxygen. Otherwise, the function should return `false`.
- **isOxygenBoiling.** This function should return the `bool` value `true` if the temperature stored in the `temperature` field is at or above the boiling point of oxygen. Otherwise, the function should return `false`.
- **isWaterFreezing.** This function should return the `bool` value `true` if the temperature stored in the `temperature` field is at or below the freezing point of water. Otherwise, the function should return `false`.
- **isWaterBoiling.** This function should return the `bool` value `true` if the temperature stored in the `temperature` field is at or above the boiling point of water. Otherwise, the function should return `false`.

Write a program that demonstrates the class. The program should ask the user to enter a temperature and then display a list of the substances that will freeze at that temperature and those that will boil at that temperature. For example, if the temperature is -20 the class should report that water will freeze and oxygen will boil at that temperature.

17. Cash Register

Design a `CashRegister` class that can be used with the `InventoryItem` class discussed in this chapter. The `CashRegister` class should perform the following:

1. Ask the user for the item and quantity being purchased.
2. Get the item's cost from the `InventoryItem` object.
3. Add a 30% profit to the cost to get the item's unit price.
4. Multiply the unit price times the quantity being purchased to get the purchase subtotal.
5. Compute a 6% sales tax on the subtotal to get the purchase total.
6. Display the purchase subtotal, tax, and total on the screen.
7. Subtract the quantity being purchased from the `onHand` variable of the `InventoryItem` class object.

Implement both classes in a complete program. Feel free to modify the `InventoryItem` class in any way necessary.

Input Validation: Do not accept a negative value for the quantity of items being purchased.

18. A Game of 21

For this assignment, you will write a program that lets the user play against the computer in a variation of the popular blackjack card game. In this variation of the game,

two six-sided dice are used instead of cards. The dice are rolled, and the player tries to beat the computer's hidden total without going over 21.

Here are some suggestions for the game's design:

- Each round of the game is performed as an iteration of a loop that repeats as long as the player agrees to roll the dice, and the player's total does not exceed 21.
- At the beginning of each round, the program will ask the users whether they want to roll the dice to accumulate points.
- During each round, the program simulates the rolling of two six-sided dice. It rolls the dice first for the computer, and then it asks the user if he or she wants to roll. (Use the `Die` class that was demonstrated in this chapter to simulate the dice).
- The loop keeps a running total of both the computer and the user's points.
- The computer's total should remain hidden until the loop has finished.
- After the loop has finished, the computer's total is revealed, and the player with the most points without going over 21 wins.

19. Trivia Game

In this programming challenge you will create a simple trivia game for two players. The program will work like this:

- Starting with player 1, each player gets a turn at answering five trivia questions. (There are a total of 10 questions.) When a question is displayed, four possible answers are also displayed. Only one of the answers is correct, and if the player selects the correct answer he or she earns a point.
- After answers have been selected for all of the questions, the program displays the number of points earned by each player and declares the player with the highest number of points the winner.

In this program you will design a `Question` class to hold the data for a trivia question. The `Question` class should have member variables for the following data:

- A trivia question
- Possible answer #1
- Possible answer #2
- Possible answer #3
- Possible answer #4
- The number of the correct answer (1, 2, 3, or 4)

The `Question` class should have appropriate constructor(s), accessor, and mutator functions.

The program should create an array of 10 `Question` objects, one for each trivia question. Make up your own trivia questions on the subject or subjects of your choice for the objects.

Group Project

20. Patient Fees

1. This program should be designed and written by a team of students. Here are some suggestions:
 - One or more students may work on a single class.
 - The requirements of the program should be analyzed so each student is given about the same workload.

- The parameters and return types of each function and class member function should be decided in advance.
 - The program will be best implemented as a multi-file program.
2. You are to write a program that computes a patient's bill for a hospital stay. The different components of the program are

The `PatientAccount` class

The `Surgery` class

The `Pharmacy` class

The main program

- The `PatientAccount` class will keep a total of the patient's charges. It will also keep track of the number of days spent in the hospital. The group must decide on the hospital's daily rate.
- The `Surgery` class will have stored within it the charges for at least five types of surgery. It can update the charges variable of the `PatientAccount` class.
- The `Pharmacy` class will have stored within it the price of at least five types of medication. It can update the charges variable of the `PatientAccount` class.
- The student who designs the main program will design a menu that allows the user to enter a type of surgery and a type of medication, and check the patient out of the hospital. When the patient checks out, the total charges should be displayed.